

COMP 2019 Final Group Report

Project Title

Prediction of freshly harvested FFB mass from images and depth data

24 April 2026

Group 2 Team Members

Name	StudentID	Username
Sophie Tejasukmana Onuki	20718618	hcyso2
Ong Wei Xuan	20617520	hfywo2
Adrian Lee Joon Yin	20611251	hfyal10
Florence Tinevimbo Chigwida	20701680	hcyfc1
Navya Binoy Valiyakunnel	20699501	hcynv2
Phung Yu Jie	20618300	hfyyp4

Supervisor

Dr. Iman Li Yiao

Client

Dr. Hazreen Haizi Binti Harith

Abstract

This project presents the development of an AI-based system for estimating the volume and mass of oil palm Fresh Fruit Bunches (FFBs) using RGB-D data. The system integrates object detection, depth-based processing, geometric modelling, and a user interface into a complete end-to-end pipeline. RGB and depth data are captured using an Intel RealSense camera and processed through a YOLO-based detection model to localise the fruit. A depth-based masking approach is then applied to isolate the fruit region, followed by single-view point cloud generation. Due to limitations in data availability and camera calibration, ellipsoid fitting is used as a robust geometric approximation method for volume estimation, which is subsequently converted into mass using a density-based model. The system is designed for deployment on edge devices and includes a real-time dashboard for visualisation and data management. Experimental results show that the system achieves an accuracy of 73% on a limited dataset, demonstrating its feasibility despite constraints in data quantity and hardware performance. The project also addresses practical challenges such as noisy depth data, hardware limitations, and integration complexity. Overall, the proposed system provides a cost-effective and scalable approach for automated FFB analysis, with potential for further improvement through enhanced datasets, calibration methods, and full system automation.

Table of Contents

1. Introduction.....	5
1.1 Purpose.....	5
1.2 Client	5
1.3 Project Description	5
2. Literature Review	5
2.1 Background Research.....	5
2.2 Related Works / Existing Solutions	7
3. Methodology.....	8
3.1 User Interface Development Methodology	8
3.2 AI Pipeline Methodology	8
4. Requirements Specification	9
4.1. Functional User Requirements	9
4.2 Functional System Requirements	10
4.3. Non-Functional System Requirements	10
4.4 Use Case Diagram	11
5. UI Design	11
5.1 Home Page	11
5.2 Dashboard Page	12
5.3 System Information Page	12
6. Implementation Description	16
6.1 UI Implementation Decisions.....	16
6.2 AI Pipeline Implementation Decisions	16
7. Problems Faced.....	20
7.1 Edge Device Integration and Hardware Constraints	20
7.2 Data Acquisition and Data Limitations	21
7.3 Object Detection and Segmentation Challenges	21
7.4 Mask Generation and Preprocessing Limitations	21
7.5 Point Cloud Reconstruction and Alignment Issues	22
7.6 Volume and Mass Estimation Constraints	22
7.7 Pipeline Integration and System Complexity	22
7.8 Software Dependencies and Compatibility Issues	22
7.9 Data Privacy and Platform Constraints	23
7.10 Stream and Colour Conversion Issues.....	23
8. Summary of Achievements	23

9. Future Work.....	23
10. Reflective Comments.....	24
11. Springtime Plan	24
References	27
Appendix A: Additional Figures / Supporting Images.....	28
Appendix B: Test Cases	31
Appendix C: Meeting Minutes.....	38
Appendix D: User Manual.....	39

1. Introduction

1.1 Purpose

The purpose of this project is to develop an intelligent system that can automatically estimate the mass of oil palm Fresh Fruit Bunches (FFB) in real time. Manual weighing is slow, labour-intensive and unsuitable for large scale harvesting operations. Instead of predicting mass directly with a deep learning model, the system captures RGB and depth data using an edge device and reconstructs a 3D representation of the FFB. The estimated volume is then converted to mass using density values taken from our client, Dr. Hazreen's dissertation. Once a prediction is generated, a user-friendly interface allows users to access, edit, delete and import FFB records with ease.

1.2 Client

The Client of this project is Dr. Hazreen Haizi binti Harith, a professor in the Faculty of Engineering at the Universiti Putra Malaysia. This collaboration was established in response to the need for a more efficient and accurate method of estimating FFB mass using AI-based techniques. Recognising that this challenge required understanding beyond traditional engineering approaches, Dr. Hazreen initiated the collaboration for our Software Engineering Group Project.

1.3 Project Description

In the oil palm industry, accurate estimation of Fresh Fruit Bunch (FFB) weight is essential for yield forecasting, harvest planning, and operational management, yet current manual weighing methods remain slow, labour-intensive, and unsuitable for increasingly automated plantation environments. Although previous research has explored image-based estimation techniques, many rely on 2D RGB information alone, limiting their ability to capture the true geometry of FFB and resulting in inconsistent performance under real plantation conditions. Challenges such as natural variations in bunch morphology, irregular shapes, lighting dependency, and the need for real-time, field-deployable systems further restrict the effectiveness of existing solutions. To address these limitations, this project proposes an AI-based approach that leverages RGB-D data from an Intel RealSense depth camera to extract both visual and geometric features for automated mass, and potentially volume, prediction. Building on this concept, the project aims to collect RGB-D samples, develop detection and segmentation modules, generate 3D point-cloud representations, train machine-learning models for estimation, and integrate these components into an edge-device pipeline with an accompanying user interface. The final deliverables include the full AI pipeline, deployed prototype, and supporting documentation, offering a more efficient and scalable alternative to manual FFB weighing in modern plantations.

2. Literature Review

2.1 Background Research

According to Mohd Kassim et al. (2012), estimating the physical properties of agricultural products, such as the weight and volume of oil palm Fresh Fruit Bunches (FFB), is critical for yield estimation and harvesting efficiency. Traditional manual weighing methods are labour-

intensive, time-consuming, and unsuitable for integration with current automated harvesting systems and they lead to inconsistencies in judgement and delayed processing, which reduces oil quality (Azhan, n.d.; Mohammadmahdi Naghipour et al., 2024; Pamornnak et al., 2017). Recent advancements in computer vision (CV), machine learning (ML), RGB-D imaging, and edge computing offer promising alternatives for accurate, non-contact mass estimation. RGB-D imaging combines regular colour (RGB) images with depth information to create 3D representations of objects. The depth data records how far each point is from the camera, allowing researchers to estimate an object's size and shape (Kumar, 2025). In agricultural studies, cameras such as the Intel Realsense series and the Microsoft Kinect 2.0 are used due to their ability to capture infrared and depth data at a low cost (Kadir et al., 2022; Pamornnak et al., 2017). The cameras are mounted in a top-down position of around 1.0m to 1.5m in height over a conveyor belt or loading ramp to obtain a clear view of the FFB (Azhan, n.d.; Pamornnak et al., 2017).

The depth videos are imported to either Intel RealSense Viewer or CloudCompare to extract point clouds and compute volume using tools like 2.5D Volume Calculation function (Azhan, n.d.). This information can then be processed on the spot by an edge device to estimate the volume and, using known density values, the mass of Fresh Fruit Bunches. Previous studies have shown that RGB-D imaging provides accurate, non-contact estimation results (Muhammad Aqil, n.d.). Computer Vision and Machine Learning (ML) are central to automating FFB mass estimation. Frames from the video data are extracted and pre-processed with noise removal via median filters, alignment of RGB and depth channels, and thresholding (Pamornnak et al., 2017; Zolfagharnassab et al., 2022). The frames are then segmented and the FFBs "mask image" are detected from the background with techniques like the Otsu method, Canny algorithms, and morphological operations (erosions/dilation) (Lai et al., 2023; Pamornnak et al., 2017). Researchers extract features such as average hue for maturity, volume index for mass, and texture via Histogram of Gradients (HOG) (Chen et al., 2022; Pamornnak et al., 2017).

The models mainly used for processing include Faster R-CNN for high accuracy detection and the YOLO series (v3, v4, v5, and v10) for real time speed (Chen et al., 2022; Kadir et al., 2022; Mohammadmahdi Naghipour et al., 2024). However, the implementation of an RGB-D-based system for oil palm Fresh Fruit Bunch estimation presents several challenges and limitations. Environmental factors such as varying lighting conditions, shadows, reflections, and complex plantation backgrounds often affect RGB image quality, leading to inaccuracies in segmentation and detection. Weather conditions like rain, fog, and humidity can further reduce visibility and interfere with both RGB and depth sensors. From a data and modelling perspective, the lack of large, well annotated RGB-D datasets for oil palm limits model generalization, while pre trained models (e.g., YOLO, Mask R-CNN) trained on generic datasets may perform poorly without fine-tuning. Small or imbalanced datasets also increase the risk of overfitting. Depth data introduces additional challenges, including sensor noise, limited range performance under sunlight, misalignment between RGB and depth frames, and surface reflectivity issues that affect accurate 3D reconstruction. Hardware and computation constraints pose further limitations, as edge devices such as the Raspberry Pi 4 have limited

processing power, higher susceptibility to thermal stress, and must balance performance with low power consumption (G52GRP Final Group Report, 2022).

Implementation difficulties include achieving real-time detection and volume estimation on low-power devices, maintaining stable system integration between components, and ensuring regular calibration for consistent accuracy. Finally, algorithmic challenges such as occlusion of FFBs by foliage, effective fusion of RGB and depth data, and discrepancies between laboratory and field performance remain significant barriers to achieving robust, reliable deployment in real-world plantation environments (Ng et al., 2024; Zolfagharnassab et al., 2022). Another common issue is the imperfect alignment between RGB and depth channels, which can affect precision. Overlapping bunches or fruitlets obscured by leaves or branches make accurate counting and 3D reconstruction difficult (Ng et al., 2024; Tang et al., 2020). Variable lighting affects RGB data reliability, while noise in depth maps can be caused by low-light conditions or reflective surfaces (Minarni Shiddiq et al., 2024; Pamornnak et al., 2017; Zolfagharnassab et al., 2022). Models trained on one plantation may struggle with variations in fruit geometry and palm varieties elsewhere (Pamornnak et al., 2017; Woittiez et al., 2017).

2.2 Related Works / Existing Solutions

Several existing studies and systems have explored the use of computer vision and RGB-D technologies for agricultural automation and object classification. A notable example is the Semi-Automated Sorting System for Germinated Oil Palm Seeds (G52GRP Final Group Report, 2022), which implemented RGB imaging and computer vision algorithms to detect and classify germinated seeds. The system demonstrated how vision-based automation can enhance sorting accuracy and efficiency in agricultural workflows. However, it primarily relied on 2D RGB data, limiting its ability to perform volumetric or mass-related analysis. Similarly, research by (Zolfagharnassab et al. 2022) utilized thermal imaging for classifying oil palm fresh fruit bunches (FFBs) based on maturity levels, showcasing the potential of non-invasive sensing methods. More recently, (Ng et al. 2024) proposed a Multiview-based classification approach using RGB imaging and deep learning to improve accuracy through multi-angle data capture. Beyond these, RGB-D cameras such as the Intel RealSense have gained traction in agricultural research for 3D reconstruction, volume estimation, and morphological analysis. Studies, including those discussed by, have demonstrated the use of RGB-D data for estimating

mass and volume of agricultural products through depth-based analysis. Nonetheless, these approaches often depend on high-performance computing or cloud-based processing, limiting their scalability and field deployment. As such, integrating RGB-D imaging with edge computing represents a promising direction for achieving real-time, on-site FFB detection and mass estimation in plantation environments. Other systems like the "Maturity Table" for digital FFB tracking and Simple Volume Integration Schemes (SVIS) for Kinect-based volume estimation have been developed (Muhamad Saufi et al., 2011). YOLOv4 typically offers higher accuracy (98.5% mAP), whereas YOLOv5 is superior in detection speed (11.4 ms), making it better for real-time harvesting robots (Chen et al., 2022) Current gaps include the lack of large datasets, the need for more robust edge deployment in uncontrolled environments,

and difficulties in modelling the irregular geometries of dense fruit bunches (Azhan, n.d.; Muhamad Saufi et al., 2011; Tang et al., 2020).

3. Methodology

3.1 User Interface Development Methodology

The User Interface (UI) was developed as an independent component, running in parallel with the AI pipeline, and edge device connectivity throughout the project. Requirement analysis was conducted at the start of the project, and full details of the identified requirements are provided in the Requirements Specification section. The UI applied the Extreme Prototyping model to rapidly design, evaluate and refine the visual aspects of the system. This allowed the interface to progress quickly and independently, ensuring that design work did not slow down while the more complex AI modules were still being developed.

I. Stages in Development

The UI development followed the three stages of Extreme Prototyping. In Stage 1, the team created static HTML and CSS mock-ups to define the overall layout, navigation flow, and main interface components such as the upload area, prediction display, and history panel. These visual-only pages allowed fast feedback and easy revisions without waiting for backend development. In Stage 2, the UI was connected to simulated backend responses, enabling it to display placeholder outputs like sample bounding boxes, segmentation colours, and example mass estimates. This helped the team refine interactions, test usability, and identify design issues while the AI modules were still in progress. Stage 3 focused on integrating the user interface with the operational backend once the edge device and RealSense camera detection were successfully established. An SQL database was also integrated to securely store estimation results for future retrieval and analysis (Hull, 2020).

3.2 AI Pipeline Methodology

I. Overview of The AI Pipeline

The objective of the AI pipeline is to estimate the volume and mass of oil palm Fresh Fruit Bunches (FFBs) using RGB-D data. The proposed approach combines object detection with depth-based geometric analysis to extract spatial information from images captured using an Intel RealSense camera. Due to practical constraints encountered during development, full 3D reconstruction from multiple viewpoints was not adopted. These constraints included the lack of reliable camera extrinsic parameters and inconsistent viewpoint alignment. Instead, a geometric approximation approach based on ellipsoid fitting was selected. This method enables stable and computationally efficient estimation of object volume using partial observations. Although multiple frames were captured from different viewpoints to improve spatial coverage, each observation was processed independently.

II. Methodological Framework

The methodology follows a structured pipeline in which RGB and depth data are used for complementary purposes. RGB images are utilised for object localisation, while depth maps provide metric distance information required for spatial analysis. This separation allows efficient use of both data modalities. Object detection is performed using a pretrained YOLO-

based model, which outputs bounding boxes defining the approximate location of the FFB. These bounding boxes are not intended for precise segmentation but are instead used to guide subsequent depth-based processing. Depth masking is then applied within the detected region to reduce background interference while preserving relevant spatial data. The masked depth information is interpreted as a representation of the object's surface geometry. Using camera intrinsic parameters, depth values are projected into 3D space to form a single-view point cloud. Each point cloud represents a partial observation of the object from a single viewpoint, avoiding the need for multi-view registration and alignment. To estimate the overall shape of the FFB, a geometric approximation model is applied. An ellipsoid is selected as the underlying representation due to its suitability for approximating organic, rounded structures. The principal directions of the point distribution are identified, and the spread of points along these directions is used to estimate the ellipsoid's dimensions. To improve robustness, statistical measures are used instead of extreme values, reducing the influence of noise, missing depth data, and residual background points. Point clouds with insufficient data are excluded to maintain stability in the estimation process.

III. Volume and Mass Estimation Strategy

The volume of the FFB is computed from the parameters of the fitted ellipsoid, providing a consistent geometric approximation without requiring full 3D reconstruction. Mass estimation is subsequently derived using a predefined density value based on empirical measurements provided by the project client. While this approach does not yield exact physical measurements, it ensures consistency, repeatability, and robustness under varying data conditions. The methodology therefore prioritises practical applicability and computational efficiency over precise geometric reconstruction. The estimated geometric and physical properties are structured for integration with the overall system. These outputs support downstream processes, including storage, visualisation, and user interaction, enabling effective interpretation of the results.

4. Requirements Specification

4.1. Functional User Requirements

Edge device (Field Operator)

- 4.1.1. The user shall capture RGB and depth images or videos using the Intel RealSense Camera.
- 4.1.2 The system shall automatically process the captured data and estimate the FFB weight.
- 4.1.3. The user shall see a simple indication that the estimation has been completed (e.g. "Prediction 17.40 kg").
- 4.1.4. The system shall work offline.
- 4.1.5. The user shall be able to connect the Odroid to the Desktop to send results.

AI Model

- 4.1.6. The AI model shall accept both RGB and Depth inputs of FFB samples.
- 4.1.7. The model shall output the predicted weight.
- 4.1.8. The model shall handle missing or partially corrupted RGB or depth data.

4.1.9. The trained model shall be exportable in a lightweight format suitable for Odroid deployment.

4.1.10. The model shall accept input data from video (.bag) frames.

Desktop Visualization (Viewer/Analyst)

4.1.11. The user shall be able to view predicted weights sent from the edge device.

4.1.12. The system shall display timestamps for each prediction.

4.1.13. The user shall be able to upload .bag files to use for estimation.

4.1.14. The user shall be able to browse the history of past estimations.

4.1.15. The user shall be able to export results in CSV format.

4.1.16. The user shall be able to rename or delete result files.

4.1.17. The desktop interface shall be simple and intuitive for non-technical users.

4.1.18. The system shall clearly notify users of any errors or connection issues with the edge device.

4.2 Functional System Requirements

Edge device (Odroid)

4.2.1. The system shall capture RGB and depth images from the Intel RealSense Camera.

4.2.2. The system shall allow uploads of .bag files as means of estimation

4.2.2. The system shall preprocess and feed the input data to the trained AI model.

4.2.3. The AI model shall locally predict the FFB's weight and volume.

4.2.4. The system shall store prediction data (weight, timestamp, Image ID) and transmit it to the desktop.

4.2.5. The device shall function without internet connectivity.

4.2.6. the system shall handle missing or misaligned data gracefully.

Desktop Visualisation

4.2.7. The user shall be able to connect the Odroid to a desktop to send results.

4.2.8. The application shall display the predicted weight and timestamp.

4.2.9. The user shall be able to view the corresponding captured images.

4.2.10. The application shall allow data export in CSV format.

4.2.11. The interface shall indicate system connection status (e.g., Connected/Disconnected).

4.3. Non-Functional System Requirements

Edge Device (Odroid)

4.3.1. The system shall generate predictions within (response time).

4.3.2. Prediction accuracy shall be within $\pm 5\%$ of the ground-truth weight.

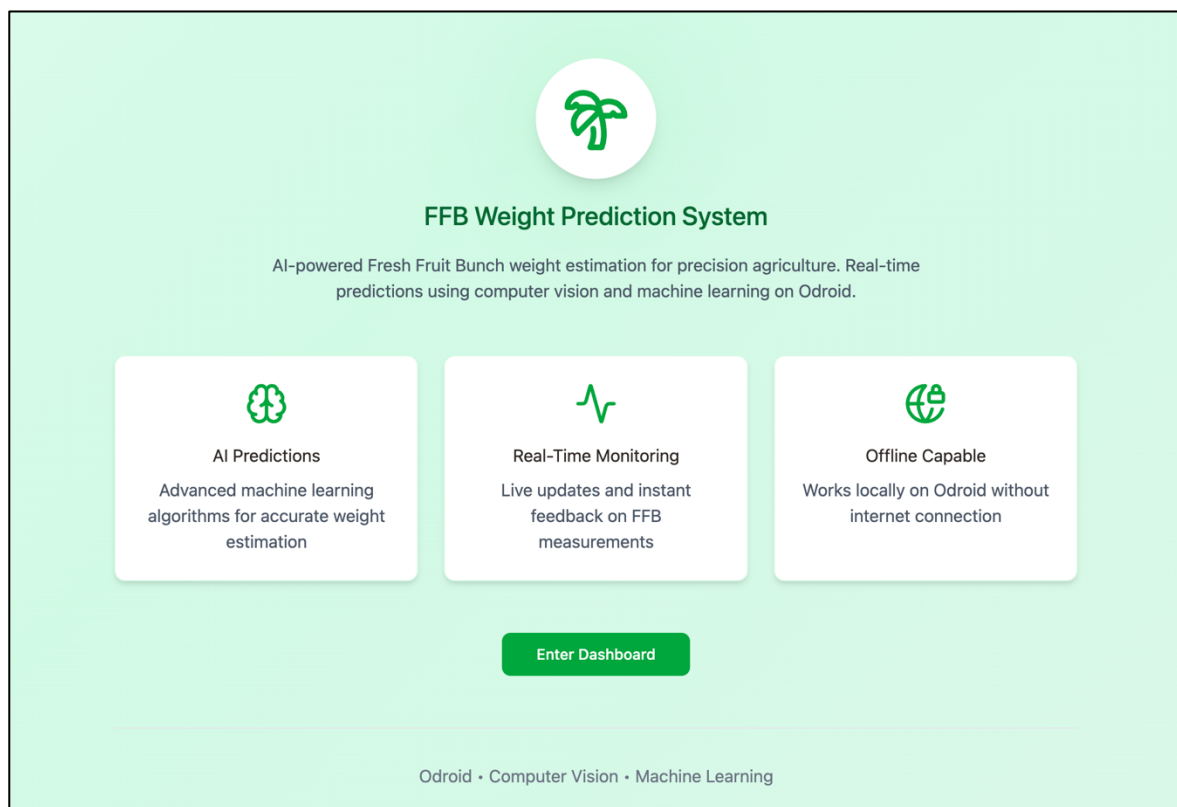
4.3.3. The device shall temporarily store data if no desktop connection is available.

4.3.4. The system shall automatically recover after power interruptions.

AI Model

4.3.5. The model shall be trained on a dataset containing at 50 (at present 6) FFB samples with ground-truth weights.

4.3.6. The model shall perform robustly under varying lighting and background conditions.



The home screen provides an overview of the FFB Weight Prediction System and its key capabilities. It presents three core features AI Predictions, Real-Time Monitoring, and Offline Operation each displayed as an interactive card to guide the user. The interface is designed with a clean, minimal layout to ensure clarity and ease of navigation. A central “Enter Dashboard” button directs users to the main system workspace, where data upload and prediction functionalities become accessible.

5.2 Dashboard Page

The dashboard (see Figure A1) presents real-time FFB analysis results, displaying the latest predicted weight alongside the corresponding RGB image used for processing (see Figure 1). A histogram with the average mass distribution per frame is also shown after uploading or capturing the depth and RGB data of the FFB (see Figure 2). The captured image includes a bounding box highlighting the detected FFB fruit for further analysis. A timestamp is also provided to indicate when the prediction was generated. In addition, a live camera feed is available, allowing users to monitor the input in real time (see Figure 3). This interface allows users to monitor FFB measurements instantly and provides a clear, organised presentation of key outputs from the AI pipeline. The settings panel allows users to configure key operational parameters of the system (see Figure 4). Options include adjusting the camera resolution, enabling or disabling automatic saving of predictions, and selecting the number of historical records to retain. A connection indicator at the top confirms active communication with the Odroid device. The prediction history panel displays previously recorded FFB weight estimations, including the predicted weight, captured image, and timestamp for each entry (see Figure 5). Users can export the full history as a CSV file or clear all stored records. This feature supports easy review, logging, and analysis of past predictions.

5.3 System Information Page

The System Information Page contains System Overview, Technical Architecture, Hardware Specifications, Model Performance and Use Case benefits. The System Overview section provides a summary of the FFB Weight Prediction System, outlining its purpose, capabilities, and deployment context (see Figure 6). It highlights the use of computer vision and machine learning for real-time FFB weight estimation and emphasizes the system’s ability to operate fully offline on Odroid hardware, making it suitable for remote agricultural environments. The Technical Architecture Overview outlines the core components of the system’s technical architecture (see figure 7). It highlights four key modules: Image Capture, AI Processing, Local Storage, and Real-Time Updates. Together, these elements describe how the system collects RGB-D data, processes it using machine learning models, stores results locally on the Odroid, and delivers live prediction updates to the dashboard. The layout provides a clear summary of how data flows through the system. The Model Performance section outlines the intended performance targets for the final system (see Figure 8). These values, average weight accuracy (± 0.5 kg), inference time (< 2 seconds), and overall model accuracy (73%) represent projected goals rather than results from the current prototype. They serve as design benchmarks for future development once the prediction model and full data pipeline are fully implemented. The Use Case and Benefits section highlight the practical applications and advantages of the FFB Weight Prediction System (see Figure 9). It outlines key use cases such as harvest optimization,

yield forecasting, quality control, and data analytics. These benefits demonstrate how reliable weight estimation can support decision-making, improve consistency across harvesting operations, and enable long-term performance tracking.

Figure 1: Latest Predicted Weight and Volume

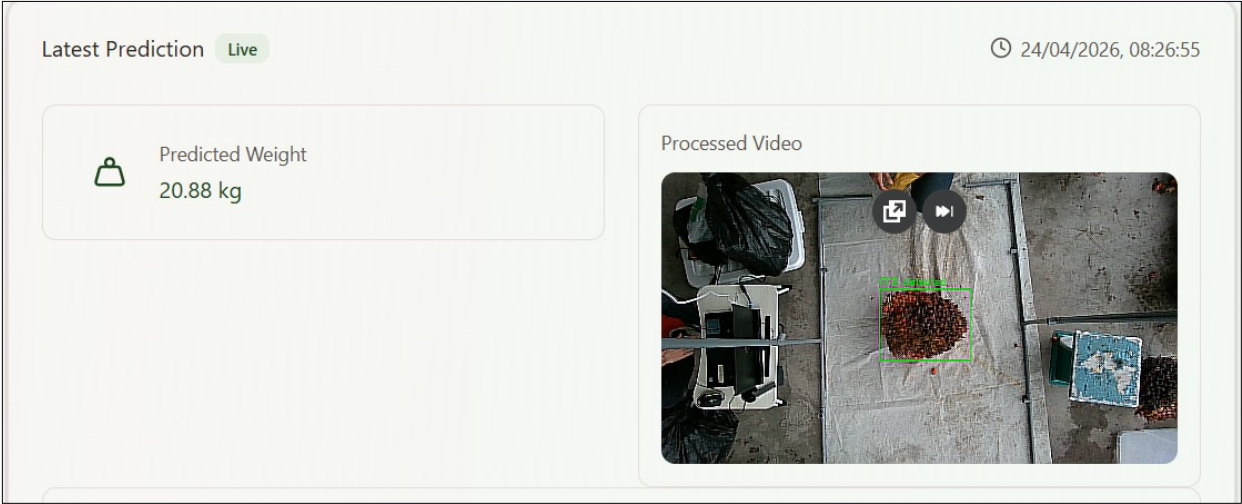


Figure 2: Histogram

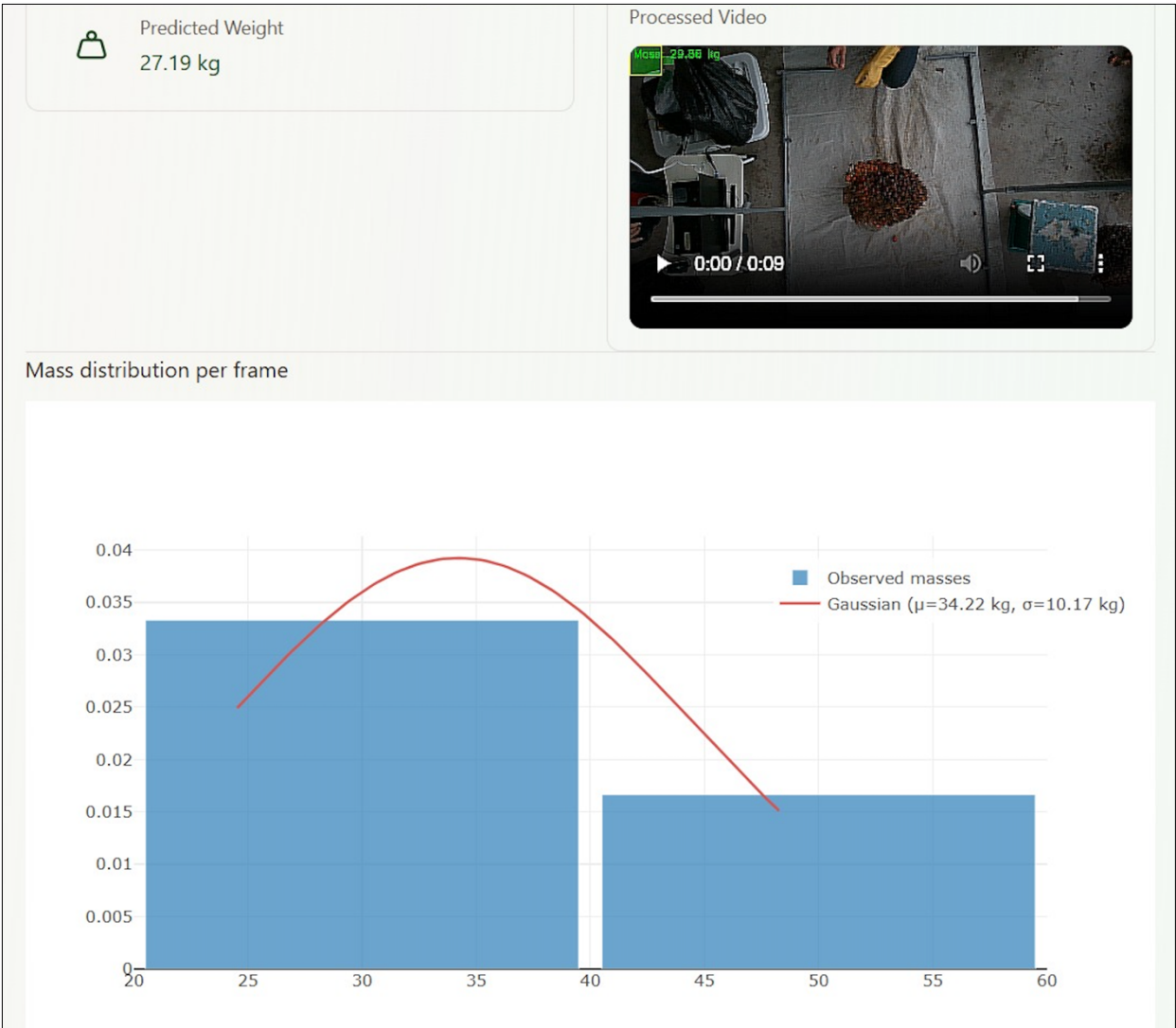


Figure 3: Live Feed in Dashboard

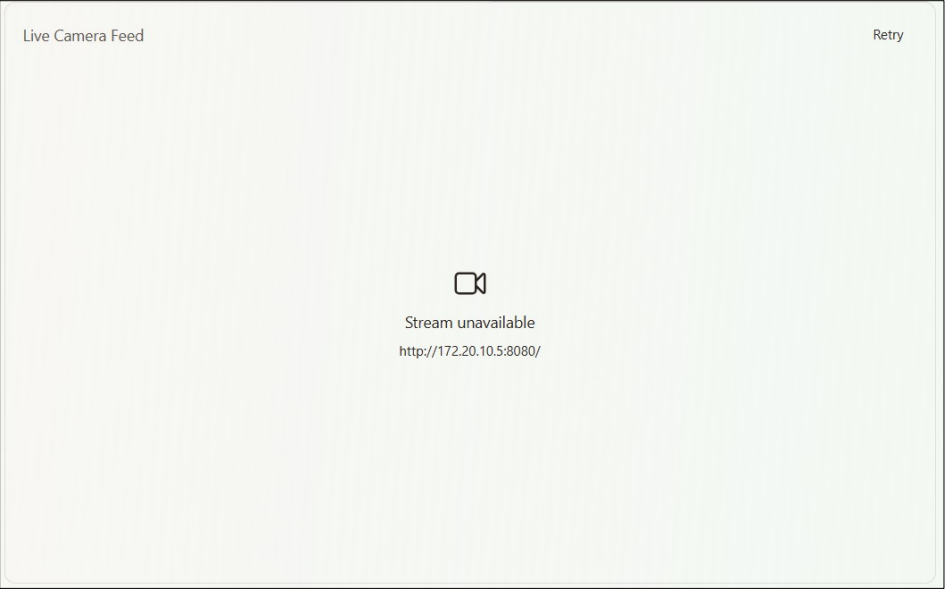


Figure 4: System Setting Panel

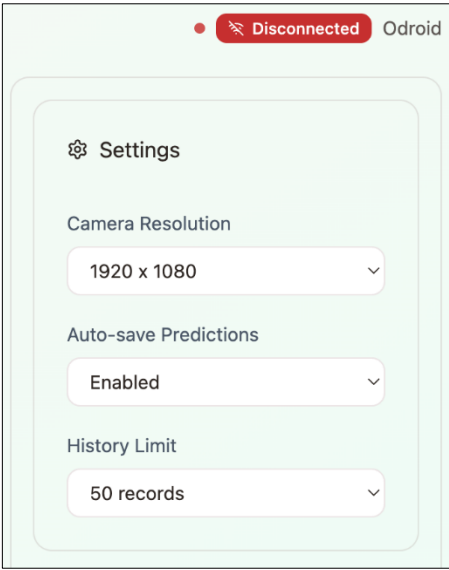


Figure 5: Prediction History

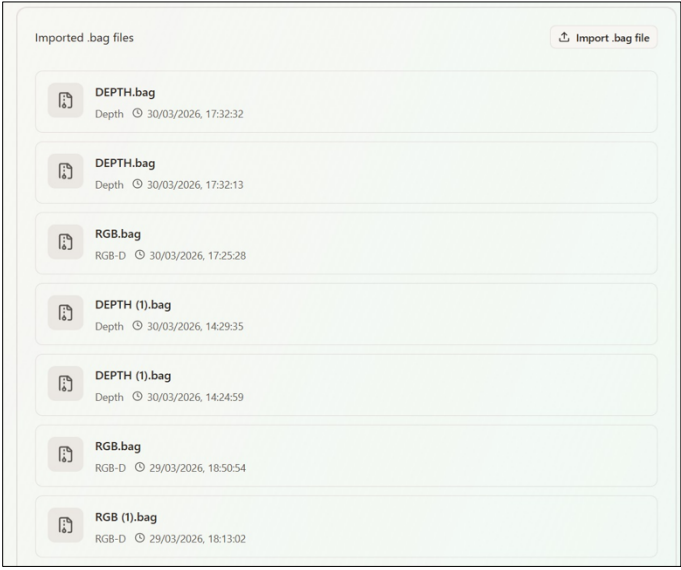


Figure 6: System Overview

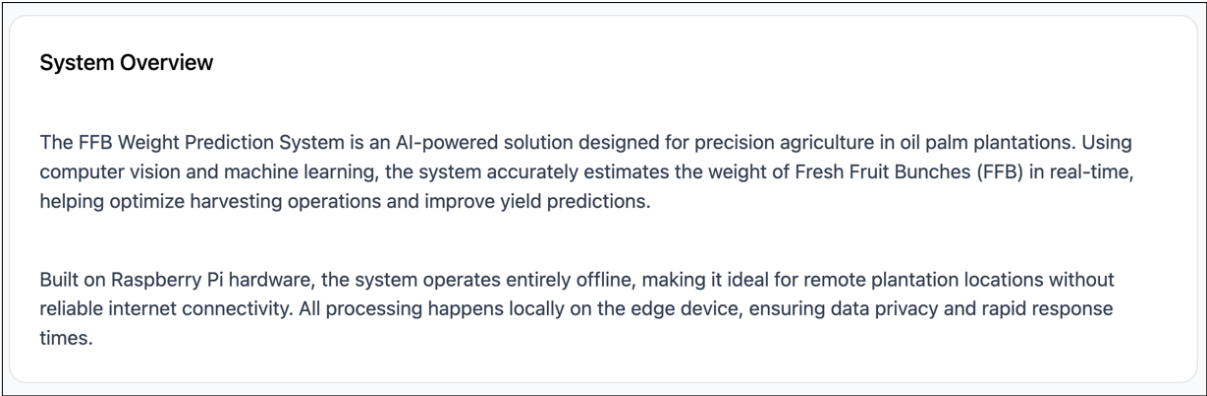


Figure 7: Technical Architecture

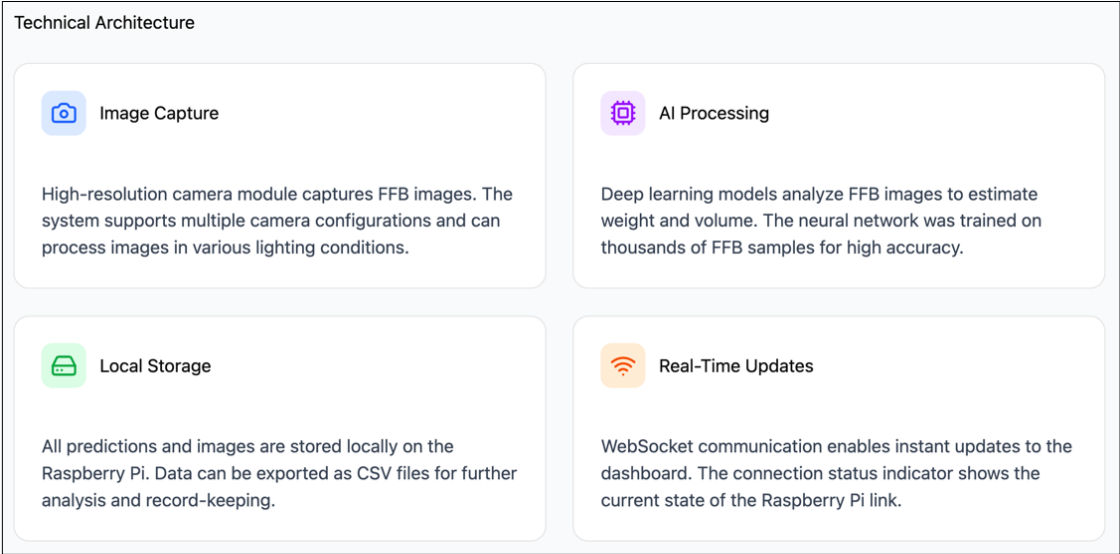
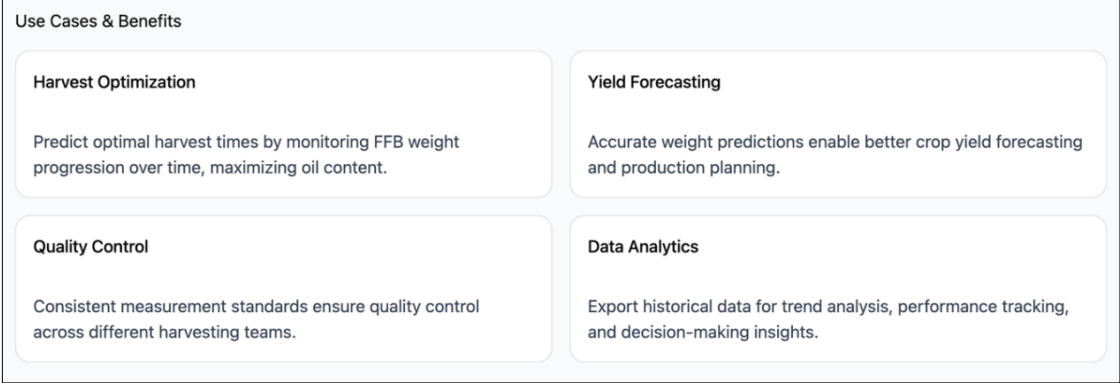


Figure 8: Model Performance



Figure 9: Use Case and Benefit



6. Implementation Description

6.1 UI Implementation Decisions

The user interface for the FFB Weight Prediction System was built using HTML, CSS, React, and Node.js, chosen for their flexibility and suitability for real-time, browser-based applications. HTML and CSS define the layout and visual styling, with `global.css` ensuring consistent colours, spacing, and typography across all screens. React's component-based structure allows the UI to be divided into modular elements such as the Cover Page, Dashboard, Latest Prediction Panel, and Prediction History, making the interface easy to update and able to re-render automatically when new data is received. Node.js provides lightweight backend support for communicating with the Odroid, managing prediction records, and enabling features like CSV export. Together, these technologies produce a clean, maintainable, and responsive interface capable of supporting real-time system monitoring. The resulting design and component integration are shown in Section 5: UI Design (Oleksandr Hutsulyak, 2024). MySQL was used as the database language to handle all of the imports that were coming into the system.

6.2 AI Pipeline Implementation Decisions

The AI pipeline for estimating the volume and mass of Fresh Fruit Bunches (FFBs) was implemented based on the proposed methodology, with several practical adaptations made to address real-world constraints. Key limitations such as limited dataset availability, absence of camera extrinsic calibration, noisy depth measurements, and edge device performance constraints significantly influenced the final implementation design. As a result, the pipeline prioritised robustness, computational efficiency, and feasibility over full geometric reconstruction accuracy. Instead of relying on complex multi-view reconstruction techniques, which require precise calibration and multiple viewpoints, the system adopts a single-view, frame-based processing approach. In this approach, each frame is processed independently and later aggregated to produce a stable final estimate. This decision reduced computational overhead and avoided error accumulation associated with misaligned point clouds and inaccurate transformations between frames.

The pipeline begins with object detection, where a YOLO-based model is used to localise the FFB within each frame. This is followed by a depth-based masking step to isolate the fruit region from the background, reducing the influence of irrelevant data during subsequent processing. From the segmented region, depth information is used to generate a partial point cloud representation of the fruit. Due to noise and incomplete data, especially around object boundaries and occluded regions, further geometric simplification is applied. To estimate volume, an ellipsoid fitting method is used as a robust approximation of the FFB shape. While this approach does not capture all geometric irregularities, it provides a computationally efficient and stable estimation under limited data conditions. The estimated volume is then converted into mass using a density-based model derived from existing literature. To improve consistency, predictions from multiple frames are aggregated, reducing the impact of outliers and noise in individual frames.

Overall, the implemented pipeline reflects a balance between theoretical design and practical constraints. While certain ideal components, such as full 3D reconstruction with accurate calibration, were not feasible within the project scope, the chosen approach ensures a working, deployable system capable of producing reasonable estimations. This highlights the importance of adapting theoretical models to real-world limitations in applied AI system development.

I. Data Preprocessing and Synchronisation

RGB-D video recordings were collected using an Intel RealSense camera, which outputs RGB frames (rgb_00xx) and corresponding depth-aligned frames (depth_00xx). The extracted RGB frames are shown in Figure 10, while the corresponding depth frames are presented in Figure 11. Each frame index corresponds to a matching timestamp (e.g., rgb_0005 $\leftrightarrow$ depth_0005), ensuring correct pairing between colour and depth information. Temporal synchronisation was performed to maintain alignment between RGB and depth streams. However, geometric alignment using camera extrinsic parameters was not implemented due to the absence of reliable calibration data. Instead, the pipeline relies on region-based processing and geometric approximation, which reduces dependency on precise pixel-level alignment while maintaining robustness.

Figure 10: Extracted RGB frames (rgb_00xx) from the RealSense recording.

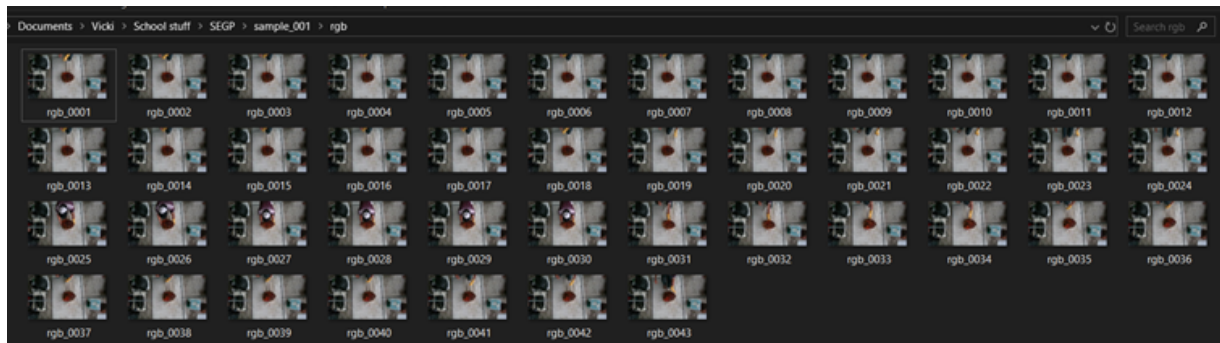
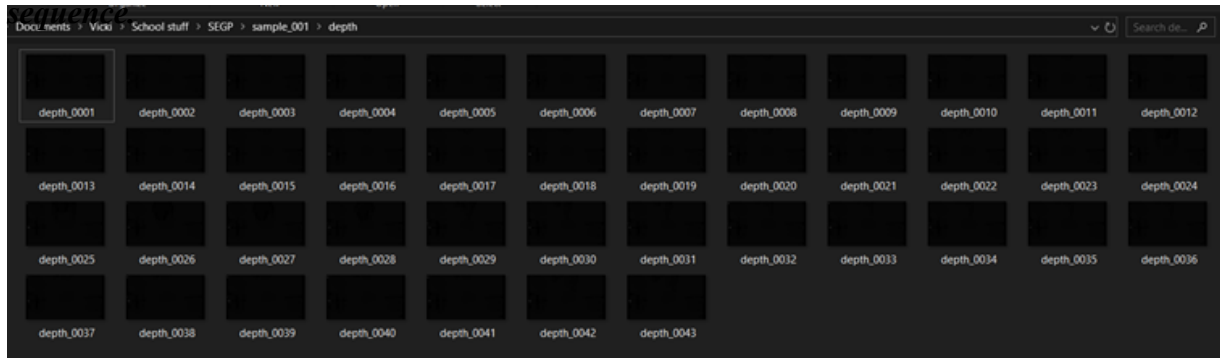


Figure 11: Extracted aligned depth frames (depth_00xx) corresponding to the RGB



II. Fruit Detection and Region Localisation

To isolate the Fresh Fruit Bunch (FFB) from background clutter, object detection was performed on RGB frames using a YOLOv8-based model. The detection output, illustrated in Figure A2, provides bounding boxes that localise the fruit within each frame. The model was deployed using an ONNX-based inference pipeline, enabling efficient execution on CPU-

constrained edge devices. A low confidence threshold was applied to maximise detection recall, particularly in cases of partial occlusion or varying lighting conditions. From multiple detections, only the highest confidence bounding box was selected per frame to ensure consistency and prevent overlapping regions from affecting downstream processing. Each detected bounding box is mapped to its corresponding depth frame using frame index matching, ensuring that spatial localisation in RGB is correctly transferred to depth processing.

III. Depth-Based Masking for Segmentation

Following object detection, depth-based masking was applied to refine segmentation within the detected region. For each bounding box, depth values were analysed to estimate a representative surface depth of the fruit as shown in Figure 12. A narrow depth range around this value was then selected, and only pixels within this range were retained. This approach removes background elements such as the supporting surface and surrounding environment while preserving the fruit structure. The resulting binary mask isolates the approximate 3D surface of the fruit and significantly reduces noise in subsequent processing. To support validation, each generated mask is overlayed onto the corresponding RGB frame, as shown in Figure A3. In this visualisation, retained fruit pixels remain unchanged while background regions are suppressed, allowing quick verification of segmentation quality. Rather than relying on complex filtering techniques, this region-based masking strategy provides a computationally efficient and robust solution under varying capture conditions.

Figure 12: Any depth pixel outside the bounding box mask is set to zero

```
# Point cloud
depth_masked = depth.copy()
depth_masked[mask == 0] = 0
points = depth_to_pointcloud(
    depth_masked, fx, fy, cx, cy, depth_scale, STEP)
```

IV. Single View Point Cloud Generation

Each masked depth frame is converted into a 3D point cloud using the RealSense camera intrinsic parameters and the pinhole camera projection model, as shown in Figure 13. Only valid depth pixels within the masked region are projected into 3D space, producing a partial representation of the fruit surface for each frame. To improve efficiency and simplify deployment, point cloud generation was implemented using NumPy-based vectorised operations rather than external libraries such as Open3D. This reduces dependency overhead and improves compatibility with edge devices while maintaining sufficient accuracy for the scale of the application. Each frame produces an independent point cloud stored in .ply format. Multi-view fusion was not performed due to the lack of camera extrinsic parameters and inconsistent overlap between viewpoints. This avoids alignment errors and ensures stable processing across frames.

Figure 13: Projection of masked depth frames into 3D point clouds. Valid pixels are mapped via RealSense intrinsics and depth scaling to produce (x, y, z) coordinates of the fruit surface.

```
def depth_to_pointcloud(depth, fx, fy, cx, cy, depth_scale, step=2):
    h, w = depth.shape
    vs, us = np.mgrid[0:h:step, 0:w:step]
    zraw = depth[vs, us]
    valid = zraw > 0
    z = zraw[valid].astype(np.float64) * depth_scale
    u = us[valid].astype(np.float64)
    v = vs[valid].astype(np.float64)
    x = (u - cx) * z / fx
    y = (v - cy) * z / fy
    return np.column_stack([x, y, z])
```

V. Geometric Approximation Using Ellipsoid Fitting

Due to incomplete surface observations from single-view point clouds, ellipsoid fitting was adopted as a robust geometric approximation method, as shown in Figure 14. Each point cloud is processed independently using Principal Component Analysis (PCA), where the covariance matrix of the point distribution is decomposed to determine the principal axes of the ellipsoid. Axis lengths are estimated using percentile-based statistics instead of extreme values, reducing sensitivity to noise and outliers. This approach provides a stable approximation of the fruit's overall shape, even when data is incomplete or partially occluded (Jolliffe, 2002). Frames with insufficient valid data are excluded to prevent unstable estimates, ensuring consistency across the pipeline.

Figure 14: PCA ellipsoid fitting implementation. Point clouds are aligned to principal axes and scaled via percentile statistics for outlier resistance. Semi-axes (a, b, c) are derived to compute fruit volume and mass, bypassing frames below the MIN_POINTS threshold.

```
def ellipsoid_mass_from_points(points):
    if len(points) < MIN_POINTS:
        return None

    center = points.mean(axis=0)
    X = points - center
    cov = np.cov(X.T)
    eigvals, eigvecs = np.linalg.eigh(cov)

    order = np.argsort(eigvals)[::-1]
    eigvecs = eigvecs[:, order]
    proj = X @ eigvecs

    a = np.percentile(np.abs(proj[:, 0]), PERCENTILE)
    b = np.percentile(np.abs(proj[:, 1]), PERCENTILE)
    c = np.percentile(np.abs(proj[:, 2]), PERCENTILE)

    if 2 * c < 0.03:
        c = max(a, b) * 0.6

    volume = (4.0 / 3.0) * np.pi * a * b * c * 5
    mass = volume * DENSITY_KG_PER_M3
    return mass, volume, (a, b, c)
```

VI. Volume and Mass Estimation

The volume of each FFB is calculated using the parameters of the fitted ellipsoid. Since only partial surface data is available, an empirical scaling factor is applied to compensate for underestimation caused by missing geometry. Mass estimation is then derived by multiplying the adjusted volume with a constant density value obtained from empirical studies provided by the project client. Although this assumes uniform density, it provides a practical approximation suitable for the available data. To improve robustness, volume and mass estimates are computed for each frame and aggregated using statistical measures such as the mean or median. This reduces the impact of noise, occlusion, and frame-specific errors.

VII. Output Generation and System Integration

The outputs of the pipeline include detection results, segmentation masks, point clouds, and estimated volume and mass values. All outputs are organised into structured directories per capture session, enabling efficient data management, debugging, and validation. Additionally, a Flask-based MJPEG streaming system was implemented to provide real-time visualisation of the input feed. The streaming process runs independently from the main pipeline, ensuring that network latency does not affect processing performance. This allows users to monitor the system remotely through a web interface (Grinberg, 2018).

7. Problems Faced

7.1 Edge Device Integration and Hardware Constraints

During system deployment, several hardware-related limitations affected the stability and performance of the edge computing setup. The Intel RealSense D435i camera initially failed to stream on the Raspberry Pi due to insufficient USB power delivery. While the sensor required up to 1.0 A during startup, the Raspberry Pi could only supply approximately 0.84 A, resulting in repeated device resets. To address this limitation, the team transitioned to an Odroid H3+, which provided more stable power delivery and improved computational capability. However, the Odroid H3+ introduced its own challenges. The device exhibited inconsistent power-on behaviour due to poor contact between the power supply and connector, which required physical reinforcement. Additionally, the initial use of Windows 11 proved unsuitable, as the Intel N6005 processor struggled to handle RealSense recording workloads efficiently. This was resolved by switching to Ubuntu Server, which offered better performance and system stability. Networking issues further complicated the setup. The Odroid lacked functional Wi-Fi drivers for the RTL8192EU chipset, and Ethernet connectivity was initially unavailable due to configuration errors in netplan. Temporary connectivity was established using manual DHCP commands before a stable static network configuration was implemented. Additional complications arose when USB drives failed to mount, delaying driver installation. Eventually, Windows Internet Connection Sharing was used to download and install the required drivers, enabling stable SSH-based access to the device.

7.2 Data Acquisition and Data Limitations

The project faced significant challenges related to data availability and quality. The dataset provided by the client contained only 11 samples instead of the expected 50, limiting both model training and validation. Although data augmentation techniques such as controlled rotations and visual transformations were applied to increase variability, the lack of real-world diversity restricted the model's ability to generalise across different fruit sizes, shapes, lighting conditions, and maturity levels. In addition, the RGB-D data captured using the RealSense camera exhibited noise and inconsistencies, particularly around object boundaries and reflective surfaces. The depth measurements were highly sensitive to camera placement and capture distance, where even small variations resulted in significant accuracy differences. Furthermore, the dataset lacked camera extrinsic parameters, making it difficult to accurately align multiple viewpoints. Combined with the limited capture angles, this prevented the construction of a complete and geometrically consistent 3D representation of the fruit. As a result, the system relied on heuristic-based assumptions, such as symmetry and estimated thickness, which reduced overall accuracy.

7.3 Object Detection and Segmentation Challenges

Several issues were encountered in the object detection and segmentation stages of the pipeline. The YOLOv8 model demonstrated sensitivity to input format variations, where identical frames produced inconsistent detection results depending on preprocessing and encoding. For example, detections failed on raw RGB arrays but succeeded when processed as PNG images, indicating a dependency on colour representation and data handling. In multi-object scenarios, the model frequently detected fruits located at the edges of the frame rather than the intended central fruit. This introduced errors in subsequent masking and segmentation stages. To mitigate this issue, a centre-based filtering strategy was implemented to prioritise detections closest to the image centre. Bounding boxes generated by the model were sometimes inaccurate, failing to tightly enclose the fruit and occasionally including background elements such as soil or surrounding vegetation. This led to imperfect segmentation masks and the propagation of errors into later stages of the pipeline. Additionally, repeated execution of the detection pipeline sometimes overwrote previously accurate results with lower-quality outputs. Model performance was also highly dependent on annotation quality, where inconsistent labelling reduced detection reliability.

7.4 Mask Generation and Preprocessing Limitations

The segmentation process introduced further challenges due to imperfect mask generation. Background noise was frequently included in the masks, resulting in unwanted points being incorporated into the reconstructed point clouds. Removing these outliers proved difficult, as aggressive filtering often eliminated valid portions of the fruit geometry. The preprocessing stage was highly sensitive to parameter selection, such as threshold values and filtering criteria. Achieving acceptable results required repeated manual tuning, reducing the overall robustness and scalability of the system.

7.5 Point Cloud Reconstruction and Alignment Issues

The reconstruction of 3D point clouds from depth data was affected by several technical limitations. The generated point clouds were often sparse, noisy, or fragmented, making it difficult to produce a complete representation of the fruit. When attempting to combine multiple views, misalignment occurred even after applying Iterative Closest Point (ICP) techniques. This issue was further compounded by the absence of accurate camera extrinsic parameters, which are essential for proper spatial alignment. Minor calibration inaccuracies resulted in significant distortions, and errors accumulated across the pipeline. Consequently, inaccuracies in earlier stages, such as detection and segmentation, were amplified during reconstruction.

7.6 Volume and Mass Estimation Constraints

Accurate estimation of fruit volume and mass proved challenging due to limitations in both data quality and modelling assumptions. Volume was often overestimated when background points were included in the reconstruction, while occlusions and missing depth information led to underestimation. Simplified geometric approximations, such as ellipsoid fitting and PCA-based methods, were unable to fully capture the irregular shape of Fresh Fruit Bunches. Additionally, estimating object thickness from single-view depth data was inherently unreliable. Mass estimation was further affected by error propagation from earlier stages. Inaccurate volume calculations, inconsistencies in scale conversion, and reliance on a constant density assumption contributed to unrealistic outputs. Since natural variations in fruit composition were not accounted for, the predicted mass values lacked consistency and precision.

7.7 Pipeline Integration and System Complexity

The multi-stage nature of the AI pipeline introduced significant integration challenges. Errors in one stage often propagated to subsequent stages, making it difficult to isolate root causes. The interdependencies between detection, segmentation, reconstruction, and estimation increased debugging complexity and development time. In addition, inconsistencies in file formats and data handling, such as transitions between .bag files, extracted frames, and NumPy arrays, added further overhead. The need for extensive manual parameter tuning also reduced automation and hindered scalability.

7.8 Software Dependencies and Compatibility Issues

The integration of multiple libraries introduced dependency conflicts throughout development. Installing certain packages, such as `opencv-contrib-python`, resulted in the removal or incompatibility of other critical libraries like NumPy. Compatibility issues also arose with `pyrealsense2`, particularly with Python 3.12, requiring additional tools such as Microsoft Visual C++ Build Tools and careful selection of compatible library versions. Data extraction from RealSense .bag files initially produced misaligned RGB and depth frames, which required explicit alignment using `rs.align(rs.stream.color)` to ensure consistency.

7.9 Data Privacy and Platform Constraints

The use of Roboflow for dataset management and model training introduced concerns regarding data privacy. Toward the end of the project, the client raised issues about the dataset being publicly accessible, which conflicted with ownership and confidentiality requirements. As a result, the team planned to transition to an alternative open-source solution to ensure compliance with data protection constraints.

7.10 Stream and Colour Conversion Issues

Additional issues were encountered during camera input handling. Certain versions of onnxruntime and opencv-python caused model loading failures due to incompatibilities. Furthermore, the USB camera frequently output frames in non-standard formats such as grayscale or YUV (e.g., UYVY or YUYV), instead of the expected BGR format. This resulted in distorted visual outputs, including grayscale or green–purple tinted video. Conflicts between OpenCV’s automatic colour conversion and manual conversion logic led to double processing of frames, further degrading image quality. These inconsistencies required careful handling of colour space conversions to ensure correct input for the detection pipeline.

8. Summary of Achievements

The project successfully delivered a fully functional and user-friendly system that integrates a complete end-to-end AI pipeline with a working database and real-time interface. The system is capable of processing input data and generating results in under one minute, demonstrating efficient performance despite operating on constrained edge hardware. An accuracy of 73% was achieved (Figure A.6), even though the evaluation was conducted on a limited dataset of only six samples. While this dataset size is small, the results indicate strong potential for improvement as more data becomes available. A significant achievement of this project lies in overcoming both financial and technical constraints. The team operated with limited resources, including restricted access to high-quality datasets and hardware limitations associated with the Odroid edge device. Challenges such as unstable hardware behaviour, compatibility issues, and performance bottlenecks required continuous troubleshooting and adaptation throughout development. Despite these difficulties, the team was able to design, implement, and integrate all major components into a coherent and working system. In addition to the technical outcomes, the project was recognised during presentation and demonstration stages, where the team achieved second place and received positive feedback during the open day. The feedback highlighted the clarity of the system design, the usability of the interface, and the practical relevance of the solution, reinforcing the effectiveness of the overall implementation. Overall, the project demonstrates that a practical and deployable solution can be achieved even under constrained conditions. The successful integration of the AI pipeline, database, and interface reflects strong technical execution, effective teamwork, and resilience in addressing real-world engineering challenges.

9. Future Work

While the current system demonstrates a functional and promising solution for FFB mass estimation, several improvements can be made to enhance performance, accuracy, and real-world applicability. A key area for improvement is the optimisation of edge hardware. The

system previously faced performance and compatibility limitations on the Raspberry Pi 4 and Odroid H3+. Future work should consider adopting the Raspberry Pi 5, which offers improved processing power, better efficiency, and remains cost-effective. This would enable faster processing, more stable execution, and the possibility of running more advanced models directly on the device. In addition, increased financial support would significantly benefit further development. The project was constrained by limited data samples and restricted testing conditions. With more funding, a larger and more diverse dataset can be collected, improving model robustness and generalisation across different fruit conditions. Access to better hardware and controlled testing environments would also enhance system reliability. From a technical perspective, improvements can be made to the AI pipeline. Future work should explore multi-view reconstruction techniques once proper camera calibration is available, allowing more accurate 3D modelling instead of relying on geometric approximations. Enhancing detection and segmentation, for example through instance segmentation models, could further reduce background noise and improve overall accuracy. Another important direction is full system automation. Integrating the pipeline with a Raspberry Pi-based setup and mechanical components such as a robotic arm or conveyor system would enable automated capture, positioning, and processing of FFBs. This would improve consistency in data collection and make the system more suitable for real-world deployment. Finally, the current mass estimation approach can be improved by moving beyond a constant density assumption. Incorporating a data-driven regression model trained on a larger dataset would allow more accurate and reliable predictions. Overall, with improvements in hardware, funding, modelling techniques, and automation, the system has strong potential to evolve into a scalable and practical solution for industrial use.

10. Reflective Comments

The project successfully delivered a fully functional and user-friendly system that integrates a complete end-to-end AI pipeline with a working database and real-time interface. The system is capable of processing input data and generating results in under one minute, demonstrating efficient performance despite operating on constrained edge hardware. As shown in Figure A6 an accuracy of 73% was achieved, even though the evaluation was conducted on a limited dataset of only six samples. While this dataset size is small, the results indicate strong potential for improvement as more data becomes available. A significant achievement of this project lies in overcoming both financial and technical constraints. The team operated with limited resources, including restricted access to high-quality datasets and hardware limitations associated with the Odroid edge device. Challenges such as unstable hardware behaviour, compatibility issues, and performance bottlenecks required continuous troubleshooting and adaptation throughout development. Despite these difficulties, the team was able to design, implement, and integrate all major components into a coherent and working system.

From a project management perspective, time management proved to be one of the key challenges, particularly during the first semester. Progress was initially slower due to difficulties in coordinating tasks, underestimating the complexity of system development, and balancing workload among team members. In some cases, tasks took longer than expected due to unforeseen technical issues, which highlighted a gap in initial planning and time estimation.

Additionally, differences in working styles and communication methods occasionally led to inefficiencies in task execution. However, this improved significantly in the second semester as the team adopted more structured approaches to task management and communication. Regular progress meetings were conducted multiple times throughout the week, allowing for better tracking of ongoing work and quicker resolution of issues. In addition, detailed task allocation and tracking were implemented using shared tools such as Google Sheets, which helped ensure accountability, visibility of progress, and clarity in responsibilities. This shift towards a more organised workflow allowed the team to better manage dependencies between different components of the system.

As the project progressed, the importance of effective time management and task adherence became increasingly evident. The team developed a more disciplined approach, with clearer deadlines, improved prioritisation of tasks, and more consistent follow-ups. This not only enhanced productivity but also reduced last-minute issues and integration delays, particularly during the final stages of development. Despite encountering numerous challenges throughout the project, the team consistently worked collaboratively to identify solutions and overcome obstacles. Technical problems were approached through research, experimentation, and group discussion, while team members supported one another in resolving issues across different parts of the system. This collaborative mindset helped maintain momentum even when facing setbacks. Overall, the project demonstrates that a practical and deployable solution can be achieved even under constrained conditions. The successful integration of the AI pipeline, database, and interface reflects strong technical execution, improved project management practices over time, and a resilient, adaptive team effort in addressing real-world engineering challenges.

[illegible]

References

- Flask Web Development, 2nd Edition [Book]*. (n.d.). Wwww.oreilly.com.
<https://www.oreilly.com/library/view/flask-web-development/9781491991725/>
- GeeksforGeeks. (2018, April 5). *Prototyping Model Software Engineering*. GeeksforGeeks.
<https://www.geeksforgeeks.org/software-engineering/software-engineering-prototyping-model/>
- Hull, M. (2020, July 15). *4 Types of Prototyping*. Wwww.andplus.com.
<https://www.andplus.com/blog/4-types-of-prototyping>
- Hutsulyak, O. (2024, August). Why use React for web development: 10 reasons to apply. TechMagic. <https://www.techmagic.co/blog/why-we-use-react-js-in-the-development>
- Jocher, G., et al. (2023). YOLOv8 by Ultralytics. GitHub.
<https://github.com/ultralytics/ultralytics>
- Jocher, G., et al. (2022). YOLOv5. GitHub.
<https://github.com/ultralytics/yolov5>
- Jolliffe, I. T. (2002). *Principal component analysis* (2nd ed.). Springer.
<https://doi.org/10.1007/b98835>
- Kumar, P. (2025). *What are RGB-D cameras & why RGB-D cameras are preferred in some embedded vision applications?* e-con Systems Blog. <https://www.e-consystems.com/blog/camera/technology/what-are-rgb-d-cameras-why-rgb-d-cameras-are-preferred-in-some-embedded-vision-applications/>
- Mohd Kassim, M. S., Wan Ismail, W. I., Ramli, A. R., & Bejo, S. K. (2012). Oil palm fresh fruit bunches (FFB) growth determination system to support harvesting operation. *Journal of Food, Agriculture & Environment*, 10(2), 620–625.
https://www.researchgate.net/publication/266165587_Oil_Palm_Fresh_Fruit_Bunches_FFB_growth_determination_system_to_support_harvesting_operation
- Muhammad Aqil Bin Muhammad Azhan (n.d.). *Oil Palm Fresh Fruit Bunch Mass Estimation Using RGB-Depth Data* [Unpublished PowerPoint slides]. Universiti Putra Malaysia.
- Ng, J., Liao, I. Y., Jelani, M. F., Chen, Z. Y., Wong, C. K., & Wong, W. C. (2024). *Multiview-based method for high-throughput quality classification of germinated oil palm seeds*. *Computers and Electronics in Agriculture*, 219, 109052.
<https://www.sciencedirect.com/science/article/pii/S0168169924000759>
- Oleksandr Hutsulyak. (2024, August). *Why Use React for Web Development: 10 Reasons To Apply*. Blog | TechMagic . <https://www.techmagic.co/blog/why-we-use-react-js-in-the-development>
- Sia, A. Z. H., Yoong, J. Y., Leuk, J. K. W., Ku, M., Yan, C. C., Goh, G. J.-V., & Hoh, W. K. (2022, April 28). *Semi-automated sorting system for germinated oil palm seeds* (Group report, G52GRP). University of Nottingham Malaysia. Unpublished group project report.
- Zolfagharnassab, S., Shariff, A. R. B. M., Ehsani, R., Jaafar, H. Z., & Aris, I. B. (2022). Classification of oil palm fresh fruit bunches based on their maturity using thermal imaging technique. *Agriculture*, 12(11), 1779.
<https://doi.org/10.3390/agriculture12111779>

Appendix A: Additional Figures / Supporting Images

Figure A1: Full Dashboard Page

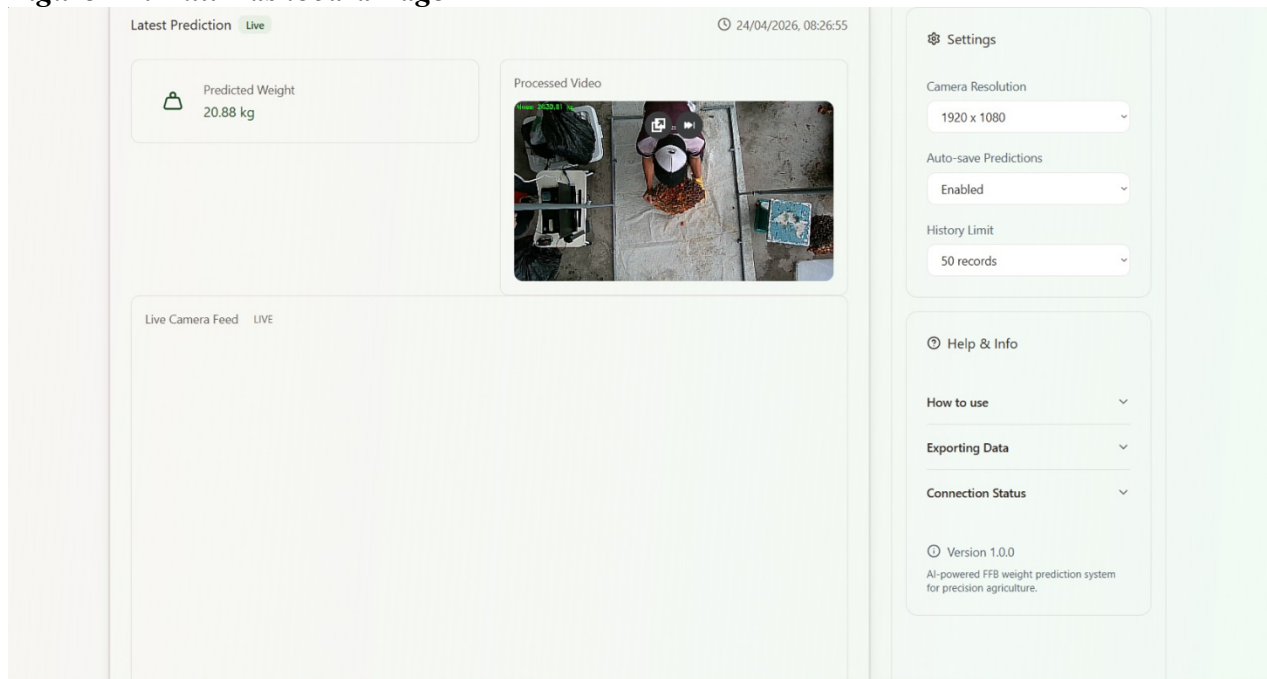


Figure A2: Detected Fresh Fruit Bunch with YOLOv8 bounding box

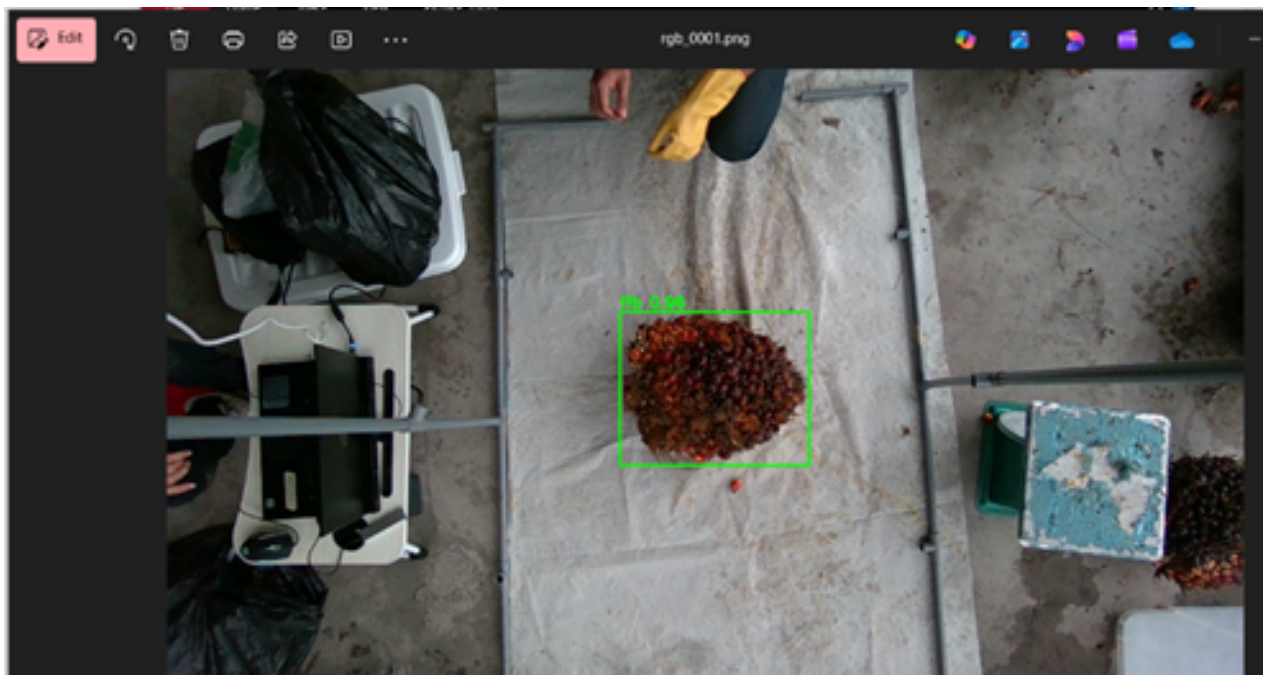


Figure A3: Overlaid segmentation masks (masks_vis) showing isolated fruit regions

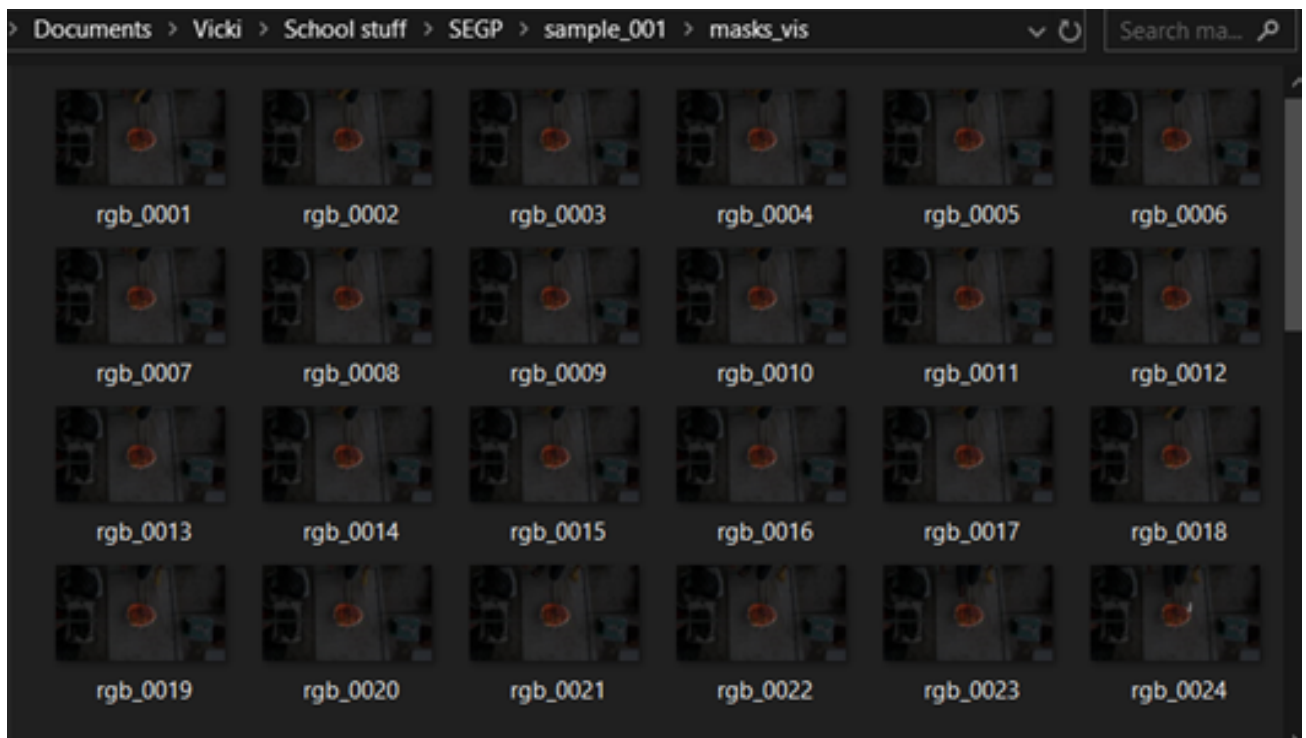


Figure A4: YOLOv8 misdetection due to PNG–RGB format differences. The RGB-array version fails to detect the fruit despite being visually identical to its PNG counterpart.

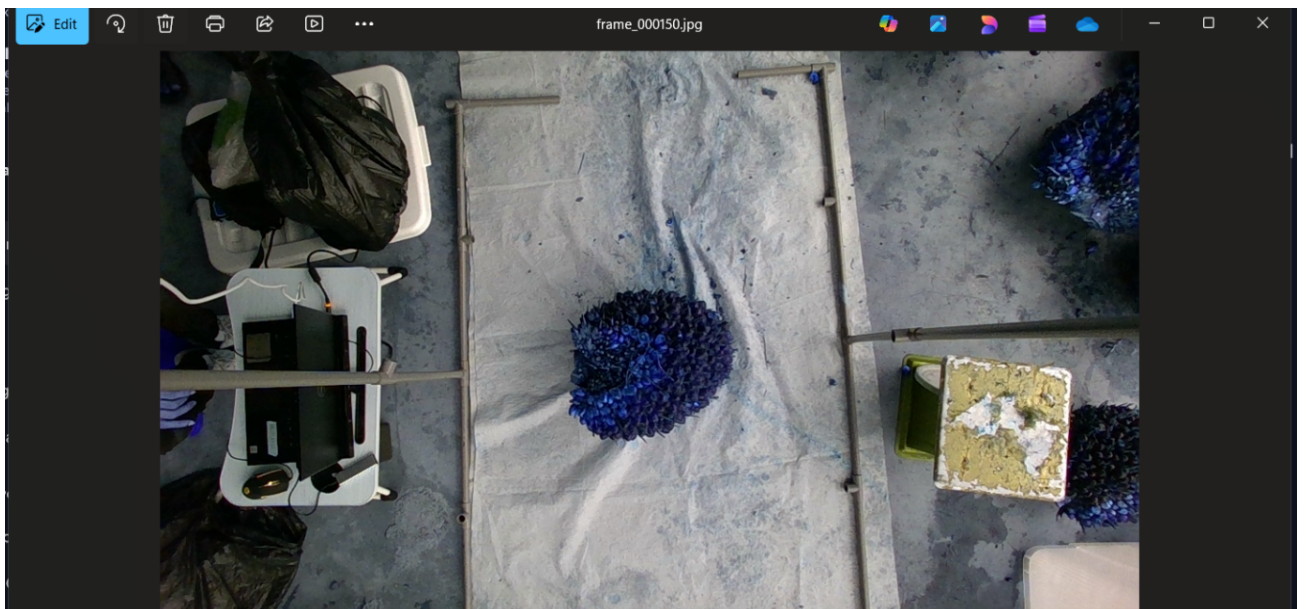


Figure A5: YOLOv8 detecting multiple FFBs in the frame, including fruit located at the far edges. This illustrates the model's sensitivity to peripheral objects, which caused incorrect selection of the target fruit before centre-based filtering was implemented.

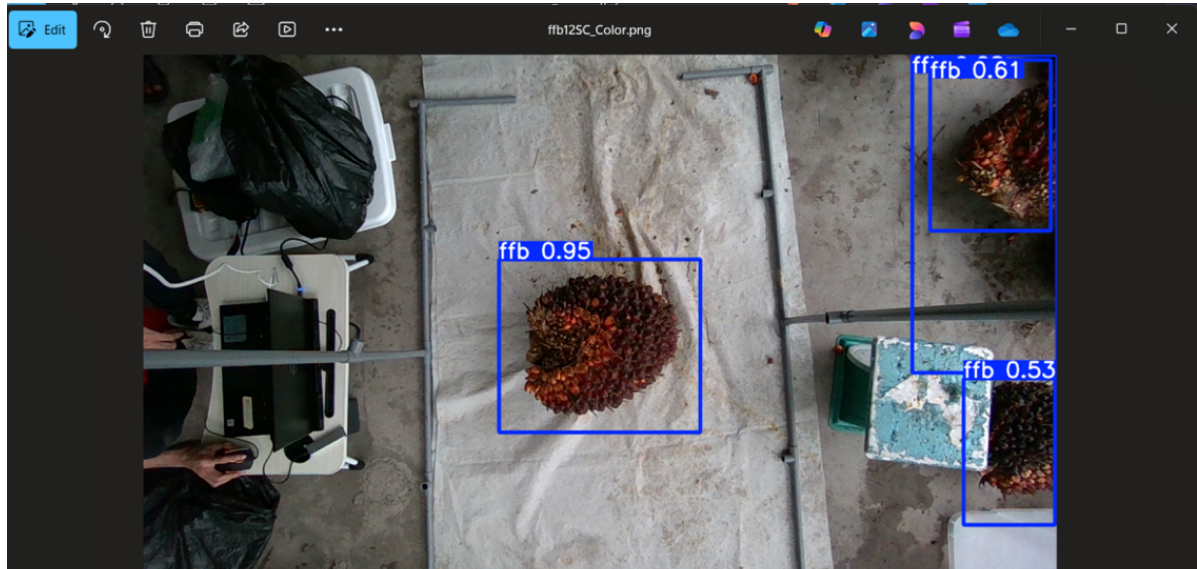
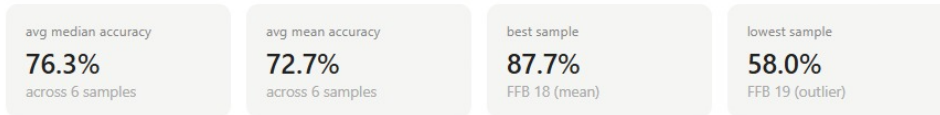


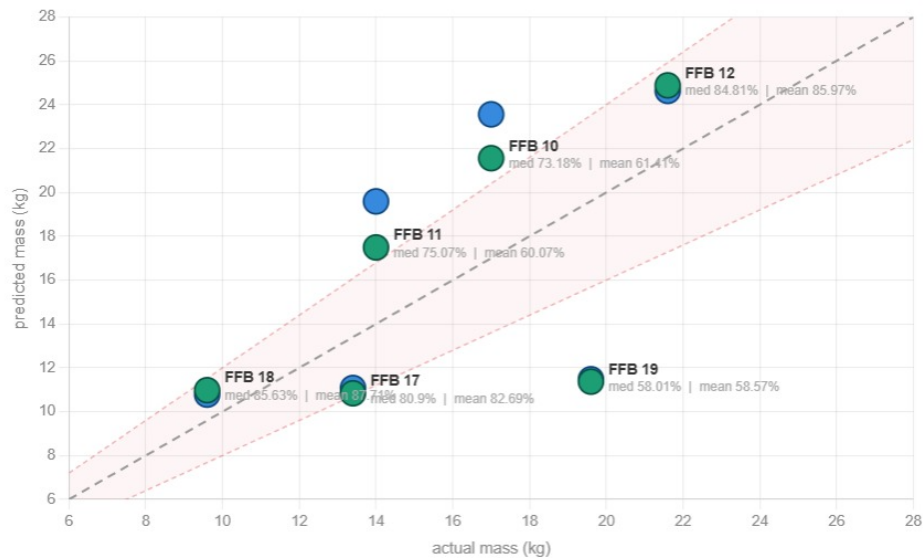
Figure A6: FFB Mass Estimation Graph Representation

FFB Mass Estimation — Predicted vs Actual

6 samples · ellipsoid fitting pipeline · Intel RealSense RGB-D



● Predicted median ● Predicted mean --- y = x (perfect prediction) ±20% error band



Appendix B: Test Cases

B.1 Testing Overview

The system was tested against the defined functional and non-functional requirements to evaluate its performance, reliability, and usability. Testing focused on key components including the edge device processing, AI model predictions, and desktop visualisation system. Each requirement was validated through structured test cases to ensure the system behaves as intended under practical conditions.

B.2 Requirement-Based Test Cases

Number/ID	1
Based on Requirement	4.1.1
Purpose of the Test: To verify if the system captures RGB and depth data using the RealSense camera.	
Pre-Requisites: RealSense camera connected to edge device.	
Steps the tester has to perform: - Start the system on the Odroid - Trigger capture process - Observe captured frames Acceptance criteria: - RGB and depth frames are captured successfully - Frames are stored with matching indices	

Number/ID	2
Based on Requirement	4.1.2
Purpose of the Test: To verify automatic processing of captured data	
Pre-Requisites: RGB-D data captured	
Steps the tester has to perform: - Run pipeline after capture Acceptance criteria: - System processes input automatically without user intervention - Prediction output is generated	

Number/ID	3
Based on Requirement	4.1.3
Purpose of the Test: Verify prediction display to user	
Pre-Requisites: Pipeline executed	
Steps the tester has to perform: - Observe UI after processing Acceptance criteria: - Prediction displayed clearly (e.g., "17.40 kg") - Display is readable and immediate	

Number/ID	4
Based on Requirement	4.1.4
Purpose of the Test: Verify offline functionality	
Pre-Requisites: No internet connection	
Steps the tester has to perform: - Disconnect network - Run system Acceptance criteria: - System runs without internet - Predictions are still generated	

Number/ID	5
Based on Requirement	4.1.5
Purpose of the Test: Verify data transmission to desktop	
Pre-Requisites: Edge device connected to desktop	
Steps the tester has to perform: - Send results from edge device Acceptance criteria: - Data successfully appears on desktop	

Number/ID	6
Based on Requirement	4.1.6
Purpose of the Test: Verify model accepts RGB-D input.	
Pre-Requisites: Valid RGB and depth frames	
Steps the tester has to perform: - Run model inference Acceptance criteria: - Model processes both inputs without error	

Number/ID	7
Based on Requirement	4.1.7
Purpose of the Test: Verify model outputs predicted weight	
Pre-Requisites: Model loaded	
Steps the tester has to perform: - Run inference Acceptance criteria: - Output contains predicted weight value	

Number/ID	8
Based on Requirement	4.1.8
Purpose of the Test:	

Verify robustness to missing data
Pre-Requisites: Corrupted or partial input
Steps the tester has to perform: - Provide incomplete RGB/depth input Acceptance criteria: - System does not crash - Output still generated or handled gracefully

Number/ID	9
Based on Requirement	4.1.9
Purpose of the Test: Verify model export format	
Pre-Requisites: Trained model available	
Steps the tester has to perform: - Export model to user computer Acceptance criteria - Data successfully exported	

Number/ID	10
Based on Requirement	4.1.10
Purpose of the Test: Verify video frame input support	
Pre-Requisites: Provide video (.bag file) input	
Steps the tester has to perform - Provide video input Acceptance criteria: - Frames processed sequentially	

Number/ID	11
Based on Requirement	4.1.11
Purpose of the Test: Verify viewing of predicted weights	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Open the desktop UI Acceptance criteria: - Predictions displayed correctly	

Number/ID	12
Based on Requirement	4.1.12
Purpose of the Test: Verify timestamp display	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform:	

- Generate prediction
- Acceptance criteria:**
- Timestamp shown with prediction

Number/ID	13
Based on Requirement	4.1.13
Purpose of the Test: Verify upload of .bag files	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Go to import/upload section and upload a .bag file of depth or RGB	
Acceptance criteria: - Upload is shown	

Number/ID	14
Based on Requirement	4.1.14
Purpose of the Test: Verify history browsing	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Go to prediction history panel	
Acceptance criteria: - Previous results accessible	

Number/ID	15
Based on Requirement	4.1.15
Purpose of the Test: Verify CSV export	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Export results	
Acceptance criteria: - CSV file generated correctly onto user laptop/computer	

Number/ID	16
Based on Requirement	4.1.16
Purpose of the Test: Verify file rename/delete	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Rename/delete result file	
Acceptance criteria: - Changes applied successfully	

Number/ID	17
------------------	----

Based on Requirement	4.1.17
Purpose of the Test: Verify UI usability	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Use UI without instructions Acceptance criteria: - UI is in intuitive and easy to navigate	

Number/ID	18
Based on Requirement	4.1.18
Purpose of the Test: Verify error notifications	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Disconnect device Acceptance criteria: - Error message displayed clearly	

Number/ID	19
Based on Requirement	4.2.3
Purpose of the Test: Verify prediction on edge device	
Pre-Requisites: Have desktop UI and Odroid working	
Steps the tester has to perform: - Run pipeline locally Acceptance criteria: - Prediction generated locally	

Number/ID	20
Based on Requirement	4.2.4
Purpose of the Test: Verify data storage	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Generate prediction Acceptance criteria: - Data stored with timestamp and ID	

Number/ID	21
Based on Requirement	4.2.6
Purpose of the Test: Verify handling of misaligned data	
Pre-Requisites: Have mismatched data	

Steps the tester has to perform:

- Provide mismatched RGB/depth data

Acceptance criteria:

- System handles without crash

Number/ID	22
Based on Requirement	4.3.1
Purpose of the Test: Verify response time	
Pre-Requisites: Have the desktop UI open and run the system	
Steps the tester has to perform: - Measure execution time	
Acceptance criteria: - Output generated < 60 seconds	

Number/ID	23
Based on Requirement	4.3.2
Purpose of the Test: Verify prediction accuracy	
Pre-Requisites: Have both ground truth and predicted weight	
Steps the tester has to perform: - Compare predicted vs actual weight	
Acceptance criteria: - Approx. 73% accuracy achieved	

Number/ID	24
Based on Requirement	4.3.3
Purpose of the Test: Verify temporary storage	
Pre-Requisites: Have desktop UI open	
Steps the tester has to perform: - Disconnect desktop	
Acceptance criteria: - Data stored locally	

Number/ID	25
Based on Requirement	4.3.4
Purpose of the Test: Verify system recovery	
Pre-Requisites: Have the desktop UI open	
Steps the tester has to perform: - Restart device after shutdown	
Acceptance criteria: - System resumes operation	

Number/ID	26
Based on Requirement	4.3.11
Purpose of the Test: Verify cross-platform compatibility	
Pre-Requisites Open the web app on Windows/macOS/Linux	
Steps the tester has to perform: - Ru UI on different OS	
Acceptance criteria: - Works on Windows/macOS/Linux	

B.3 Test Suite

Test No.	Brief Test Case Description	Pass	Fail
1	Verify if the system captures RGB and depth data using the RealSense camera.	✓	
2	Verify automatic processing of captured data	✓	
3	Verify prediction display to user	✓	
4	Verify offline functionality	✓	
5	Verify data transmission to desktop	✓	
6	Verify model accepts RGB-D input.	✓	
7	Verify model outputs predicted weight	✓	
8	Verify robustness to missing data	✓	
9	Verify model export format	✓	
10	Verify video frame input support	✓	
11	Verify viewing of predicted weights	✓	
12	Verify timestamp display	✓	
13	Verify upload of .bag files	✓	
14	Verify history browsing	✓	
15	Verify CSV export	✓	
16	Verify file rename/delete	✓	
17	Verify UI usability	✓	
18	Verify error notifications	✓	
19	Verify prediction on edge device	✓	
20	Verify data storage	✓	
21	Verify handling of misaligned data	✓	

22	Verify response time	✓	
23	Verify prediction accuracy	✓	
24	Verify temporary storage	✓	
25	Verify system recovery	✓	
26	Verify cross-platform compatibility	✓	

B.4 Example Outputs

The object detection results are shown in Appendix A and the implementation description section. As for the UI, the working prediction estimation result is shown in the UI design section.

B.5 Accuracy Evaluation

The system achieved an accuracy of 73% when evaluated against available ground-truth weights. This was based on a limited dataset of six samples, which restricts the reliability of the evaluation. Despite this limitation, the results demonstrate the feasibility of the proposed approach and indicate potential for improved accuracy with a larger dataset.

B.6 Limitations of Testing

Testing was constrained by limited data availability and hardware limitations. The small dataset size prevented comprehensive evaluation across diverse conditions, and the absence of camera calibration data limited the accuracy of geometric reconstruction. As a result, the reported performance may not fully reflect real-world deployment scenarios.

Appendix C: Meeting Minutes

A complete set of meeting minutes (Meetings 1 – 40) is available at the following repository:

[Group 2 – Meeting Minutes](#)

Due to the number of meetings, only selected summaries are included below.

Meeting 14 – Supervisor Meeting (17 November 2025)

The team presented progress on the UI and 3D reconstruction. Feedback was given on improving data storage and ensuring results are recorded for traceability. Issues with point cloud reconstruction were discussed, and the supervisor suggested using rotation estimation and feature matching techniques. It was also confirmed that a recorded demo would be acceptable. Tasks were assigned for UI integration, reconstruction improvements, and hardware troubleshooting.

Meeting 26 – Supervisor Meeting (29 January 2026)

The team reviewed feedback on the interim report and discussed areas requiring improvement, particularly in component results and overall clarity. It was confirmed that the use case diagram could remain largely conceptual, with minor adjustments such as updating “Raspberry Pi” to “Odroid.” Emphasis was placed on enhancing the user interface, as it would be a key factor during demonstration and evaluation. The importance of fully understanding the system for the

live demo and presentation was also highlighted. Hardware issues related to unstable power supply were addressed, with plans to purchase a more reliable solution. Additionally, the team was advised that multiple AI pipeline approaches could be developed in parallel and compared later, as the project is intended to demonstrate a proof of concept rather than achieve perfect accuracy.

Meeting 39 – Client and Supervisor Meeting (22 April 2026)

The team presented the developed system to the client and supervisor and received feedback on both functionality and presentation. Suggestions were made to improve the user interface, including adding graphical representations of predicted mass and confidence levels, as well as improving the visualisation of bounding boxes and their correspondence with depth data. It was also proposed to display intermediate outputs either within the interface or through an additional view for better transparency. Minor discussions included brochure preparation and administrative matters such as reimbursement for hardware costs. Overall, the feedback focused on refining the system's presentation and usability for final demonstration.

Appendix D: User Manual

A descriptive User Manual is included inside our web application to guide non-technical users on how to use the FFB Weight Prediction System. This includes an overview of our dashboard features (Figure D.1), understanding the status indicator (Figure D.2), sidebar settings (Figure D.3), and tips and best practices when using the web app (Figure D.4).

Figure D.1: Dashboard Features

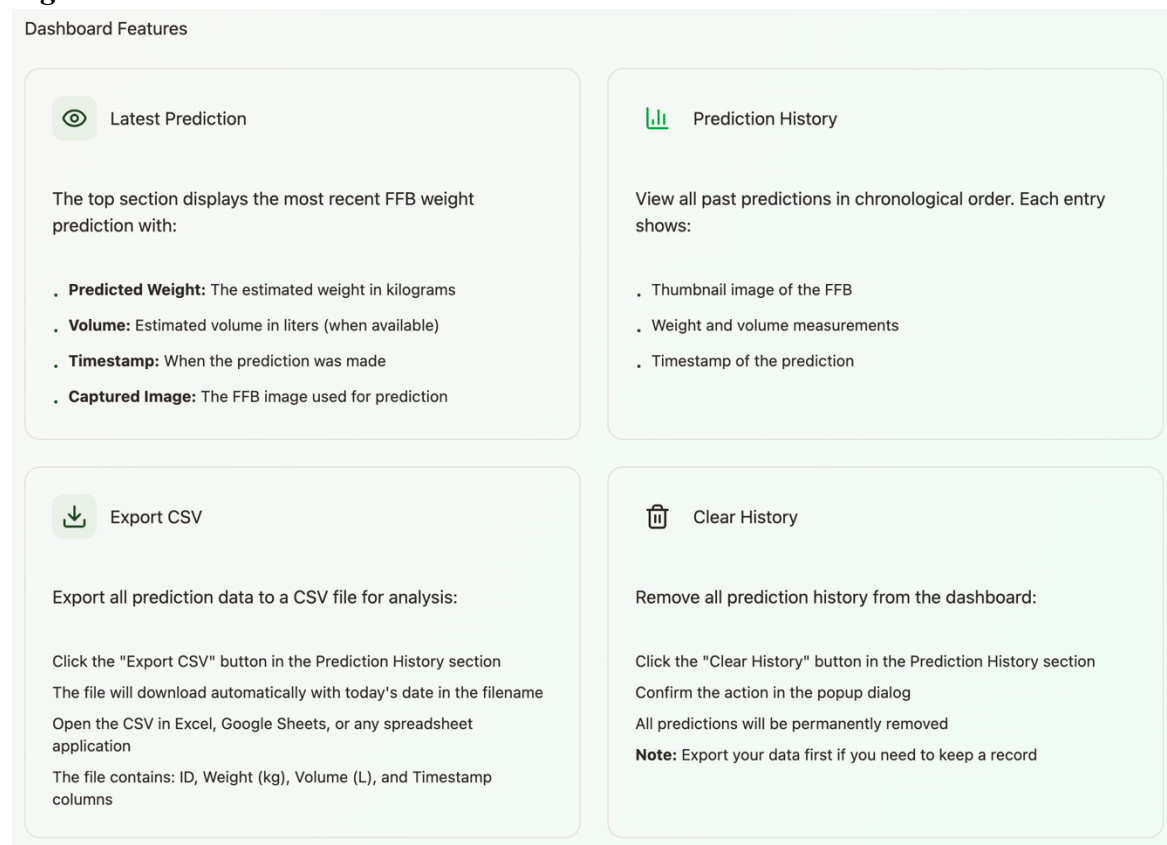


Figure D.2: Understanding the Status Indicator

Understanding the Status Indicator



Connection Status

The status indicator in the top-right corner shows the connection state to your Odroid device:

- . **Green (Connected):** The system is receiving predictions from the Odroid
- . **Red (Disconnected):** No connection to the Odroid - check network settings

Figure D.3: Sidebar Settings

Sidebar Settings



The sidebar on the right side of the dashboard contains configuration options:

- . **Camera Resolution:** Adjust the capture resolution (1920x1080, 1280x720, or 640x480)
- . **Auto-save Predictions:** Enable or disable automatic saving of predictions
- . **History Limit:** Set maximum number of records to keep (50, 100, 200, or Unlimited)

Figure D.4: Tips and Best Practices

Tips & Best Practices

Regular Data Exports

Export your prediction data regularly to maintain backups and enable long-term trend analysis.

Monitor Connection Status

Keep an eye on the connection indicator. If it shows disconnected, check your network connection and Odroid device status.

Review Latest Predictions

Check the Latest Prediction card frequently to see real-time updates from your Odroid camera system.

Use History for Analysis

The prediction history allows you to track trends over time and identify patterns in FFB weight measurements.